

LO4 Kubernetes Practical Exam Guide

Use this on your CentOS VM terminal where kubectl works, for example student@vm69-61. Use gedit only when a YAML file is required. Before starting:

```
kubectl get nodes
kubectl get all
mkdir -p ~/lo4 && cd ~/lo4
```

Many tasks use a Deployment named web. Create/reset it when needed:

```
kubectl delete deploy web --ignore-not-found
kubectl create deployment web --image=nginx:1.25 --replicas=3
kubectl get pods -l app=web
```

1. Create Deployment and export YAML

Where: Terminal

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
kubectl get deploy web -o yaml > web.yaml
kubectl get pods -l app=web
```

2. DockerHub authenticated pulls

Where: Terminal

```
kubectl create secret docker-registry dockerhub \
  --docker-server=docker.io \
  --docker-username=USER \
  --docker-password=PASS \
  --docker-email=email@example.com
kubectl patch sa default -p '{"imagePullSecrets":[{"name":"dockerhub"}]}'
kubectl get secret dockerhub -o jsonpath='{.type}'; echo
```

3. Scale two ways

Where: Terminal + YAML

```
kubectl scale deploy web --replicas=5
kubectl get rs -l app=web
kubectl get deploy web -o yaml > web.yaml
gedit web.yaml &
# change spec.replicas: 5, save, then:
kubectl apply -f web.yaml
kubectl get rs -l app=web
```

4. Rolling update

Where: Terminal

```
kubectl set image deploy/web nginx=nginx:1.27
kubectl rollout status deploy/web
kubectl get rs -l app=web
```

5. History and rollback

Where: Terminal

```
kubectl annotate deploy/web kubernetes.io/change-cause="updated nginx" --overwrite
kubectl rollout history deploy/web
kubectl rollout undo deploy/web
kubectl rollout status deploy/web
```

6. RollingUpdate maxSurge/maxUnavailable

Where: Terminal

```
kubectl patch deploy web -p '{"spec":{"strategy":{"type":"RollingUpdate","rollingUpdate":{"maxSurge":1,"maxUnavailable":0}}}}'
kubectl get deploy web -o jsonpath='{.spec.strategy}'; echo
```

7. Recreate strategy

Where: Terminal

```
kubectl patch deploy web -p '{"spec":{"strategy":{"type":"Recreate"}}}'
kubectl get deploy web -o jsonpath='{.spec.strategy.type}'; echo
```

8. CPU/memory requests and limits

Where: Terminal

```
kubectl set resources deploy web \
  --requests=cpu=100m,memory=128Mi \
  --limits=cpu=500m,memory=256Mi
kubectl describe pod -l app=web
```

9. revisionHistoryLimit

Where: Terminal

```
kubectl patch deploy web -p '{"spec":{"revisionHistoryLimit":3}}'
kubectl get deploy web -o jsonpath='{.spec.revisionHistoryLimit}'; echo
```

10. Bad image tag

Where: Terminal

```
kubectl set image deploy/web nginx=nginx:doesnotexist
kubectl rollout status deploy/web
kubectl get pods
kubectl get rs -l app=web
kubectl rollout undo deploy/web
```

11. List pods by selector

Where: Terminal

```
kubectl get pods -l app=web
kubectl get rs -l app=web
kubectl get deploy web -o jsonpath='{.spec.selector.matchLabels}'; echo
```

12. Expose Deployment

Where: Terminal

```
kubectl expose deploy web --port=80 --target-port=80
kubectl get svc web
kubectl get svc web -o yaml
```

13. Sidecar container

Where: YAML: web-sidecar.yaml

```
cat > web-sidecar.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-sidecar
spec:
  replicas: 1
  selector:
    matchLabels: {app: web-sidecar}
  template:
    metadata:
      labels: {app: web-sidecar}
    spec:
      volumes:
        - name: shared
          emptyDir: {}
      containers:
        - name: nginx
          image: nginx:1.25
          ports: [{containerPort: 80}]
          volumeMounts: [{name: shared, mountPath: /usr/share/nginx/html}]
        - name: sidecar
          image: busybox
          command: ["sh","-c","while true; do date > /data/index.html; sleep 10; done"]
          volumeMounts: [{name: shared, mountPath: /data}]
EOF
kubectl apply -f web-sidecar.yaml
kubectl get pods -l app=web-sidecar
```

14. httpd Deployment with nodeSelector

Where: YAML: httpd.yaml

```
cat > httpd.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd
spec:
  replicas: 2
  selector:
    matchLabels: {app: httpd}
  template:
    metadata:
      labels: {app: httpd}
    spec:
      nodeSelector:
        disktype: ssd
      containers:
        - name: httpd
          image: httpd:2.4
          ports:
            - name: http
              containerPort: 80
EOF
```

```
kubectl apply -f httpd.yaml
kubectl get pods
kubectl describe pod -l app=httpd
# If Pending, fix with:
kubectl label node minikube disktype=ssd
```

15. Bare Pod

Where: Terminal

```
kubectl run busy --image=busybox --restart=Never -- sleep 3600
kubectl get pod busy
kubectl delete pod busy
```

16. StatefulSet + headless Service

Where: YAML: redis.yaml

```
cat > redis.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  clusterIP: None
  selector: {app: redis}
  ports:
    - name: redis
      port: 6379
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis
  replicas: 3
  selector:
    matchLabels: {app: redis}
  template:
    metadata:
      labels: {app: redis}
    spec:
      containers:
        - name: redis
          image: redis:7
          ports:
            - name: redis
              containerPort: 6379
EOF
kubectl apply -f redis.yaml
kubectl get pods -l app=redis
kubectl run tmp --rm -it --image=busybox -- nslookup redis-0.redis.default.svc.cluster.local
```

17. DaemonSet

Where: YAML: agent.yaml

```
cat > agent.yaml <<'EOF'
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: agent
spec:
  selector:
    matchLabels: {app: agent}
  template:
    metadata:
      labels: {app: agent}
    spec:
      containers:
        - name: agent
          image: busybox
          command: ["sh", "-c", "while true; do sleep 3600; done"]
EOF
kubectl apply -f agent.yaml
kubectl get ds
kubectl get pods -l app=agent -o wide
```

18. Job

Where: Terminal

```
kubectl create job calc --image=busybox -- sh -c 'echo $((6*7))'
kubectl get jobs
kubectl logs job/calc
```

19. CronJob

Where: Terminal

```
kubectl create cronjob date --image=busybox --schedule="* * * * *" -- date
kubectl get cronjob
kubectl get jobs
kubectl patch cronjob date -p '{"spec":{"suspend":true}}'
```

20. Manual Job from CronJob

Where: Terminal

```
kubectl create job manual-date --from=cronjob/date
kubectl get job manual-date
kubectl logs job/manual-date
```

21. StatefulSet ordering

Where: Terminal

```
kubectl scale statefulset redis --replicas=3
kubectl get pods -l app=redis -w
kubectl scale statefulset redis --replicas=1
kubectl get pods -l app=redis -w
```

22. Two containers share emptyDir

Where: YAML: shared.yaml

```
cat > shared.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: shared-demo
spec:
  volumes:
    - name: shared
      emptyDir: {}
  containers:
    - name: writer
      image: busybox
      command: ["sh", "-c", "echo hello > /data/msg; sleep 3600"]
      volumeMounts: [{name: shared, mountPath: /data}]
    - name: reader
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts: [{name: shared, mountPath: /data}]
EOF
kubectl apply -f shared.yaml
kubectl exec shared-demo -c reader -- cat /data/msg
```

23. List storage

Where: Terminal

```
kubectl get sc
kubectl get pv
kubectl get pvc
```

24. ConfigMap as volume

Where: Terminal + YAML: cm-vol.yaml

```
kubectl create configmap appcfg --from-literal=color=blue --from-literal=mode=dark
cat > cm-vol.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: cm-demo
spec:
  volumes:
    - name: cfg
      configMap: {name: appcfg}
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts: [{name: cfg, mountPath: /etc/cfg}]
EOF
kubectl apply -f cm-vol.yaml
kubectl exec cm-demo -- ls /etc/cfg
kubectl exec cm-demo -- cat /etc/cfg/color
```

25. ConfigMap subPath

Where: YAML: cm-subpath.yaml

```
cat > cm-subpath.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
```

```

    name: cm-subpath
spec:
  volumes:
    - name: cfg
      configMap: {name: appcfg}
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: cfg
          mountPath: /etc/app/color.conf
          subPath: color
EOF
kubectl apply -f cm-subpath.yaml
kubectl exec cm-subpath -- cat /etc/app/color.conf

```

26. Secret as volume

Where: Terminal + YAML: secret-vol.yaml

```

kubectl create secret generic mysecret --from-literal=username=admin --from-literal=password=s3cret
cat > secret-vol.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: secret-demo
spec:
  volumes:
    - name: sec
      secret:
        secretName: mysecret
        defaultMode: 0400
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: sec
          mountPath: /etc/sec
          readOnly: true
EOF
kubectl apply -f secret-vol.yaml
kubectl exec secret-demo -- ls -l /etc/sec
kubectl exec secret-demo -- cat /etc/sec/password

```

27. StatefulSet PVC per replica

Where: YAML: redis-pvc.yaml

```

cat > redis-pvc.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: redis-pvc
spec:
  clusterIP: None
  selector: {app: redis-pvc}
  ports: [{port: 6379}]
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis-pvc
spec:
  serviceName: redis-pvc
  replicas: 3
  selector:
    matchLabels: {app: redis-pvc}
  template:
    metadata:
      labels: {app: redis-pvc}
    spec:
      containers:
        - name: redis
          image: redis:7
          volumeMounts: [{name: data, mountPath: /data}]
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
EOF
kubectl apply -f redis-pvc.yaml
kubectl get pvc
kubectl delete statefulset redis-pvc

```

```
kubectl get pvc
```

28. initContainer

Where: YAML: init.yaml

```
cat > init.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
spec:
  volumes:
  - name: data
    emptyDir: {}
  initContainers:
  - name: seed
    image: busybox
    command: ["sh", "-c", "echo seeded > /data/file"]
    volumeMounts: [{name: data, mountPath: /data}]
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "cat /data/file; sleep 3600"]
    volumeMounts: [{name: data, mountPath: /data}]
EOF
kubectl apply -f init.yaml
kubectl logs init-demo -c app
```

29. readOnly volume mount

Where: YAML: readonly.yaml

```
cat > readonly.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: readonly-demo
spec:
  volumes:
  - name: data
    emptyDir: {}
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
    volumeMounts:
    - name: data
      mountPath: /data
      readOnly: true
EOF
kubectl apply -f readonly.yaml
kubectl exec readonly-demo -- sh -c 'echo x > /data/file'
```

30. emptyDir sizeLimit

Where: YAML: sizelimit.yaml

```
cat > sizelimit.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: sizelimit-demo
spec:
  volumes:
  - name: scratch
    emptyDir:
      sizeLimit: 100Mi
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
    volumeMounts: [{name: scratch, mountPath: /scratch}]
EOF
kubectl apply -f sizelimit.yaml
kubectl describe pod sizelimit-demo
```

31. Secret from literals

Where: Terminal

```
kubectl create secret generic dbcreds --from-literal=username=admin --from-literal=password=s3cret
kubectl get secret dbcreds -o yaml
kubectl get secret dbcreds -o jsonpath='{.data.password}' | base64 -d; echo
```

32. Secret from files

Where: Terminal

```

echo cert > cert.pem
echo key > key.pem
kubectl create secret generic tlsfiles --from-file=cert.pem --from-file=key.pem
kubectl get secret tlsfiles -o yaml

```

33. TLS Secret

Where: Terminal

```

kubectl create secret tls mytls --cert=cert.pem --key=key.pem
kubectl get secret mytls -o yaml

```

34. Secret volume vs env

Where: YAML + Terminal

```

# env example inside a pod container:
env:
- name: PASSWORD
  valueFrom:
    secretKeyRef:
      name: dbcreds
      key: password
# verify existing secret volume:
kubectl exec secret-demo -- cat /etc/sec/password

```

35. envFrom secretRef

Where: YAML: envfrom.yaml

```

cat > envfrom.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: envfrom-demo
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
    envFrom:
    - secretRef:
        name: dbcreds
EOF
kubectl apply -f envfrom.yaml
kubectl exec envfrom-demo -- env

```

36. imagePullSecrets

Where: Terminal + YAML snippet

```

kubectl create secret docker-registry regcred \
  --docker-server=docker.io \
  --docker-username=USER \
  --docker-password=PASS
# add to pod/deployment spec:
spec:
  imagePullSecrets:
  - name: regcred

```

37. Mount only one Secret key

Where: Terminal + YAML: secret-one.yaml

```

kubectl create secret generic multi --from-literal=a=1 --from-literal=b=2 --from-literal=c=3
cat > secret-one.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: secret-one
spec:
  volumes:
  - name: sec
    secret:
      secretName: multi
      items:
      - key: b
        path: only-b.txt
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
    volumeMounts: [{name: sec, mountPath: /etc/sec}]
EOF
kubectl apply -f secret-one.yaml
kubectl exec secret-one -- ls /etc/sec
kubectl exec secret-one -- cat /etc/sec/only-b.txt

```

38. ClusterIP DNS

Where: Terminal

```
kubectl get deploy web || kubectl create deployment web --image=nginx --replicas=2
kubectl expose deploy web --port=80
kubectl run tmp --rm -it --image=busybox -- nslookup web.default.svc.cluster.local
```

39. NodePort

Where: Terminal

```
kubectl delete svc web --ignore-not-found
kubectl expose deploy web --type=NodePort --port=80
kubectl get svc web
minikube service web --url
curl $(minikube ip):$(kubectl get svc web -o jsonpath='{.spec.ports[0].nodePort}')
```

40. LoadBalancer

Where: Terminal 1 and Terminal 2

```
# Terminal 1:
kubectl delete svc web --ignore-not-found
kubectl expose deploy web --type=LoadBalancer --port=80
kubectl get svc web
# Terminal 2:
minikube tunnel
# Terminal 1:
kubectl get svc web
```

41. Headless Service DNS

Where: Terminal

```
kubectl apply -f redis.yaml
kubectl get svc redis
kubectl run tmp --rm -it --image=busybox -- nslookup redis.default.svc.cluster.local
```

42. Endpoints / EndpointSlice

Where: Terminal

```
kubectl get endpoints web
kubectl get endpointslices -l kubernetes.io/service-name=web
kubectl describe endpoints web
```

43. Cross-namespace FQDN

Where: Terminal

```
kubectl create ns other
kubectl run tmp -n other --rm -it --image=busybox -- wget -qO- web.default.svc.cluster.local
kubectl run tmp -n other --rm -it --image=busybox -- nslookup web
```

44. Port-forward Service vs Pod

Where: Terminal

```
kubectl port-forward svc/web 8080:80
# new terminal:
curl localhost:8080
# pod version:
POD=$(kubectl get pod -l app=web -o jsonpath='{.items[0].metadata.name}')
kubectl port-forward pod/$POD 8080:80
```

45. port / targetPort / nodePort

Where: Terminal

```
kubectl get svc web -o yaml
```

46. Debug pod connectivity

Where: Terminal

```
kubectl run tmp --rm -it --image=busybox -- sh
# inside pod:
wget -qO- web:80
nc -zv web 80
nslookup web
exit
```

47. Two Deployments, one Service

Where: YAML: shop.yaml

```
cat > shop.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
```

```

metadata: {name: shop-v1}
spec:
  replicas: 2
  selector: {matchLabels: {app: shop, ver: v1}}
  template:
    metadata: {labels: {app: shop, ver: v1}}
    spec:
      containers:
      - name: echo
        image: hashicorp/http-echo
        args: ["-text=from-v1"]
        ports: [{containerPort: 5678}]
  ---
apiVersion: apps/v1
kind: Deployment
metadata: {name: shop-v2}
spec:
  replicas: 2
  selector: {matchLabels: {app: shop, ver: v2}}
  template:
    metadata: {labels: {app: shop, ver: v2}}
    spec:
      containers:
      - name: echo
        image: hashicorp/http-echo
        args: ["-text=from-v2"]
        ports: [{containerPort: 5678}]
  ---
apiVersion: v1
kind: Service
metadata: {name: shop}
spec:
  selector: {app: shop}
  ports:
  - port: 80
    targetPort: 5678
EOF
kubectl apply -f shop.yaml
kubectl get endpoints shop
kubectl run tmp --rm -it --image=busybox -- sh -c 'for i in 1 2 3 4 5; do wget -qO- shop; done'

```

48. Diagnose Service failure

Where: Terminal

```

kubectl run tmp --rm -it --image=busybox -- nslookup web
kubectl get endpoints web
kubectl get svc web -o yaml
kubectl get pods --show-labels
kubectl describe pod -l app=web

```

49. NodePort verify

Where: Terminal

```

kubectl expose deploy web --type=NodePort --port=80 --name=web-np
kubectl get svc web-np
minikube service web-np --url

```

50. HTTP livenessProbe

Where: Terminal

```

kubectl patch deploy web --type=json -p='[{"op": "add", "path": "/spec/template/spec/containers/0/livenessProbe", "value": {"httpGet": {"path": "/health", "port": 80}}}]'
kubectl describe pod -l app=web

```

51. Redis readinessProbe

Where: YAML: edit redis.yaml

```

# Add inside redis container:
readinessProbe:
  tcpSocket:
    port: 6379
  periodSeconds: 5
# Then:
kubectl apply -f redis.yaml
kubectl describe pod -l app=redis

```

52. startupProbe

Where: YAML: edit web.yaml

```

# Add inside container:
startupProbe:
  httpGet:
    path: /
    port: 80
  failureThreshold: 30

```

```
    periodSeconds: 5
# Then:
kubectl apply -f web.yaml
kubectl describe pod -l app=web
```

53. No probes

Where: No command

Say: Without probes, Kubernetes only knows whether the container process is running. It does not know whether the application is

Cleanup if you want a fresh namespace: `kubectl delete all --all; kubectl delete pvc --all; kubectl delete cm --all; kubectl delete secret`
`dockerhub regcred dbcreds mysecret multi tlsfiles mytls --ignore-not-found`